



A Constituent college of CVM University

CLASS: SY BSc (CA & IT) - III - SEM - 2022 -23

UNIT - 3

UG02CBCA01 - OBJECT ORIENTED PROGRAMMING CONCEPTS

Unit-3 Functions, Function Overloading, Inheritance and Function Overriding

- Parameter Passing, Default Arguments, Inline Functions, Friend Functions
- Function Overloading
- Inheritance
- Role Of Access Specifies in Inheritance
- Function Overriding
- Virtual Function, Pure Virtual Function, Abstract Class

Introduction to Functions

- Dividing a program into functions is one of the major of top-down structured programming.
- Another advantage of using function is that it is possible to reduce the size of a program by calling and using them at different places in the program.
- In the function declaration,

Void swap(int x, int y);

Void: it is return type. It can be any type as per return value of function.

swap: it is function name.

int x, int y: it is parameter passing. You can pass any type of parameter.

- The function call is:

Swap();

- Syntax of function:-

```
void show( );           //function declaration
```

```
main( )  
{  
    show( );           //function call  
    .....  
}
```

```
void show( )           // function definition  
{
```

```
..... // function body  
}
```

- When the function is called, control is transferred to the first statement in the function body.
- The other statements in the function body are then executed and control returns to the main program when the closing brace is encountered.

Parameter Passing

- Parameter passing is a mechanism for communication of data and information between the calling function (caller) and the called function (callee).
- C++ supports the following three types of parameter passing schemes:
 - ✓ Pass by value
 - ✓ Pass by address
 - ✓ Pass by references (only in c++)
- The parameters used to transfer data to a function are known as input -parameters. And those used to transfer the result to the caller are known as output-parameters.
- The parameters used to transfer data in both the directions are called input-output parameters.
- Parameter can be classified as formal parameters and actual parameters.
- The formal parameters are those specified in the function declaration and function definition.
- The actual parameters are those specified in the function call.
- The following conditions must be specified for a function call.
 1. The number of arguments in the function call and the function must be the same.
 2. The data type of each of the argument in the function call should be the same as the corresponding parameter in the function declaration statement.
- However, the names of the arguments in the function call and the parameter in the function definition can be different.

Pass by value

- The default mechanism of parameter passing is called pass by value. This mechanism does not change the contents of the argument variable in the calling function (caller) even if they are changed in the called function (callee), because the content of the actual parameter in a caller is copied to the formal parameter in the callee.
- The formal parameter is stored in the local data area of the callee.
- The actual parameter is stored in the local data area of caller.

- Changes to the parameter within the function will effect only the copy (formal parameter), and will have no effect on the actual argument.

- Example:-

```
#include<iostream.h>
void swap(int x, int y)
{
    int t;                // Temparary used in swapping
    cout <<" value of X & Y before exchange:" <<x <<" "<<y<<endl;
    t=x;
    x=y;
    y=t;

    cout<<" value of X & Y after exchange:"<<x<<" "<<y<<endl;
}
void main( )
{
    int a , b;
    cout<<"Enter two integers A & B: ";
    cin>>a>>b;
    swap(a,b);
    cout<<"value of A & B after function called."<<a<<" "<<b;
}
```

- Output :-

Enter two integers A & B: 10 20

Value of X & Y before exchange: 10 20

Value of X & Y after exchange: 20 10

Value of A & B after function called: 10 20

Pass by Address:

- Instead of passing of value, the address of the variable is passed. In the function, the address of argument is copied into a memory locate instead of value.
- The de-referencing operator is used to access the variable in the called function.
- Swap integer values by pointers:-
- Example:

```
#include<iostream.h>
void swap(int *x, int *y)
{
    int t;
    t=*x;        //assign the value stored in address x to t
    *x=*y;        //assign the value stored in address y to address x
    *y=t;        //assign the value of t to address y
}
void main()
{
```

UNIT – 3 - UG02CBCA01 - OBJECT ORIENTED PROGRAMMING CONCEPTS

```
int a, b;
cout<<"enter two number";
cin>>a>>b;
swap(&a, &b);
cout<<"value of a and b"<<a<<b;
}
```

- In main(), the statement swap(&x, &y) invokes the function swap and assigns the address of the actual parameters a and b to the formal parameters x and y respectively.
- In swap(), the statement t=*x; assigns the contents of the memory location pointed to by the pointer(address) stored in the variable x (it is effectively accessing the contents of the actual variable a in the caller).
- Similarly parameter y holds the address of parameter b.

Pass by Reference:

- Passing parameters by references has the functionality of pass-by-pointer and the syntax of call-by-value.
- To pass an argument by references, the function call is similar to call by value.
- In the function declaration that parameter which are to be received by references must be preceded by the '&' operator.
- The reference type formal parameters are accessed in the same way as normal value parameters.
- Any modification to them will also be reflected in the actual parameters.
- Swap integer values by Reference.

```
#include<iostream.h>
void swap(int &x, int &y)
{
    int t;
    t = x;
    x = y;
    y = t;
}
void main( )
{
    int a , b;
    cout<<" Enter two integer A & B: ";
    cin>>a>>b;
    swap(&a, &b);
    cout<<"value of A & B after swap: "<<a<<" "<<b;
```

- output:-
Enter
two
integers
A & B: 10
20

Value of A & B after swap : 20 10

- In main(), the statement, swap(a, b); translated into swap (&a, &b); internally during the compilation.
- The function declaration void swap(int &a , int &b) indicates that the formal parameters are of reference type and hence, they must be bound to the memory location of the actual parameter.
- Thus, any access made to the reference formal parameters in the swap() function refers to the actual parameters.
- The following statement body of the swap() function:

t = x;

x = y;

y = t;

- (As treated by the compiler) have the following interpretation internally.

t = *x;

*x = *y;

*y = t;

- The following points can be noted about reference parameters:
 1. A reference can never be null, it must always refer to a legitimate object(variable).
 2. Once established, a reference can never be changed so as to make it point to a different object.
 3. A reference does not require any explicit mechanism to dereference the memory address and access the actual data value.

Default Argument

- Normally, a function call should specify all the arguments used in the function definition.
- In C++, the function may be defined to take default values of the omitted arguments by providing the default values in function prototype.
- Parameters without default arguments are placed first, and those with default value are placed later.
- The compiler looks at the prototype to see how many arguments a function uses and alerts to program for possible default values.
- The arguments specified in the function call explicitly always *override* the default values specified in the function prototype.

- Thus, if you pass value of default argument, compiler considers that value otherwise compiler considers default value.
- A function with default argument provides high flexibility on programming.
- Functions may be defined with more than one default argument.

- Example:

```
#include<iostream.h>
Void printline(char= '-', int=75);
Void main()
{
    Printline();           //use both default argument
    Printline("!");        //use 2nd argument as default
    Printline('*',45);      //ignores default arguments.
}
Void printline(char ch, int t)
{
    Int I;
    For (i=0; i<t; i++)
    {
        Cout<<ch;
    }
}
```

- Output:

```
-----
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
*****
```

Inline Functions

- One of the objectives of using functions in a program is to save some memory space, which become important when a function is to be called many times.
- Every time a function is called, it takes a lot of extra time in executing a series of instructions for tasks such as jumping to the function, saving register, pushing arguments into calling function.
- C++ has a different solution to this problem.
- To eliminate the cost of calls to small functions, c++ proposes a new feature called inline function.
- An inline function is a function that is expanded in line when it is invoked.
- That is, the compiler replaces the function call with the corresponding Function code.

- The inline function are defined as bellow:

Inline function-header

```
{
    function-body
}
```

Example:-

```
inline double cube(double a)
{
    return(a*a*a);
}
void main()
{
    double c,d;
    c=cube(3,0);
    d=cube(2.5 + 1.5);
}
```

- It is easy to make a function inline.
- All we need to do is to prefix the keyword inline to the function definition. All inline function must be defined before they are called.
- The function are made inline when they are small enough to be defined in one or two lines:
- Example:-

```
inline double cube(double a)
{
    return(a*a*a);
}
```

- The inline keyword simply sends a request, not a command, to the compiler.
- The compiler may ignore this request if the function definition is, too long or too complicated, so it compiled the function as a normal function.
- Some situations where inline expansion may not work:-
 - For functions returning values, if a loop, a switch, or goto exists.
 - For functions not returning values, if a return statement exists. i.e return NULL;
 - If function contain static variables.
 - If inline functions are recursive.
- Use of inline function :-

```
#include<iostream.h>
inline float mul(float x, float y)
```

```

{
    return(x * y);
}
inline double div(double p, double q)
{
    return(p/q);
}

int main( )
{
    float a=12.34;
    float b=9.82;

    cout<< mul(a,b)<<" \n" ;
    cout<< div(a,b)<<" \n" ;
    return 0;
}

```

Friend Functions

- The concept of encapsulation and data hiding dictate that non-member functions should not be allowed to access an object's private and protected members. The policy, if you are not a member you cannot get it.
- Sometimes, this feature leads to considerable inconvenience in programming. Imaging that the user wants a function to operate on objects of two different classes.
- At such time, it is required to allow functions outside a class to access and manipulate the private members of the class. In c++, this is achieved by using the concept of friends.
- One of the feature of c++ is allowing non-member functions to access even the private members of a class using friend functions.
- The function declaration must be prefixed by the keyword *friend* whereas function definition must not.
- The friend function could be defined anywhere in the program similar to any normal c++ function.
- A friend function possesses the following special characteristics:
 - The scope of a friend function is not limited to the class in which it has been declared as a friend.
 - A friend function cannot be called using the object of the class; it is not in the scope of the class. It can be invoked a normal function without the use if any object.

- Unlike class member functions, it cannot access the class members directly. However, it can use the object and the dot operator with each member name to access both the private and public members.
- It can be either declared in the private part or the public part of a class without affecting its meaning.
- Friendship, when applied to program design, is an escape mechanism which creates exceptions to the rule of data hiding. Usage of friend classes should, therefore, be limited to those cases where it is absolutely essential.

- Example:

```
#include<iostream.h>
class one
{
    private:
        int data1;
    public:
        void setdata(int init)
        {
            data1=init;
        }
        friend int add_both(one a, two b);
};

class two
{
    private:
        int data2;
    public:
        void setdata(int init)
        {
            data2=init;
        }
        friend int add_both(one a, two b);
};

//friend function
int add_both(one a, two b)
{
    return a.data1+b.data2;
}

void main()
{
    one a;
    two b;
    a.setdata(5);
    b.setdata(10);
    cout<<"sum of a and b is "<<add_both(a,b);
}
```

- Output:
Sum of a and b is 15
- Friend functions are useful in the following situations:
 - Function operating on objects of two different classes. This is the ideal situation where the friend function can be used to bridge two classes.
 - Friend functions can be used to increase the versatility of overloaded operator.
 - Sometimes, a friend allows a more obvious syntax for calling a function, rather than what a member function can do.

Functions Overloading:

- Overloading means to use of the same thing for different purposes.
- C++ permits overloading of functions. This means that we can use the same function name to create function that performs a variety of different tasks.
- Function overloading means many functions with same name but different argument-list or its data type.
- This is known as function polymorphism in oop.
- Using the concept of function overloading we can design a family of function with one function name but with different argument list.
- The function could perform different operations depending on the argument list in the function call.
- The correct function to be executed is determined by checking the number and type of arguments but not on the function type.
- For example, an overload function add() handles different types of data as shown bellow.
- Function Declarations

```
int add(int a, int b);  
int add(int a, int b, int c);  
double add( double x, double y);  
double add( int p, double q);  
double add( double p, int q);
```

- Function calls

```
cout << add(5, 10);  
cout << add(15, 10, 0);  
cout << add(12.5, 7.5);  
cout << add(5, 10, 15);  
cout << add(0.75, 5);
```

- A function call first matches the prototype having the same number and type of arguments and then calls the appropriate function for execution.
- The function selection involves the following steps :-
 1. The compiler first tries to find an exact match in which the types of actual arguments are the same, and use that function.
 2. If an exact match is not found, the compiler uses the integral promotions to the actual arguments such as, to find a match

Char to int
Float to double

3. When either of them fails, the compiler tries to use the built-in is unique. Conversions, to the actual arguments and then uses the function whose match is unique. If the conversion is possible to have multiple matches, then the compiler will generate an error message.

For example:

```
long square ( long n)
double square (double X)
Function call :- Square(10)
```

Will cause an error because int argument can be converted to either long or double.

4. If all of the steps fail, then the compiler will try the user-defined conversation in combinations with integral promotions and built-in conversations to find a unique match.
- **Example of function overloading:-**

```
#include<iostream.h>

int volume(int);
double volume(double, int);
long volume(long, int, int);

void main( )
{
    int a=10;
    double b= 12.5;
    long c= 100;

    cout<< Volume(a) <<" \n" ;
    cout<< Volume(b, a);
    cout<< Volume(c, a, a);
}

int volume (int s) // Cube
{
    return (s*s*s);
}
```

```

}
double volume(double r, int h) // Cylinder
{
    return(3.14*r*h);
}
long volume( long l, int b, int h) //rectangle
{
    return(l*b*h);
}

```

Introduction of Inheritance

- It allows the extension and reuse of the existing code without having to rewrite the code.
- Reusability is important feature of OOP. It is nice to reuse something that already exists. It is time saving.
- C++ strongly supports the concept of reusability. The C++ classes can be reused in several ways.
- Once a class has been written and tested, it can be adapted by other programmers to suit their requirements.
- This is basically done by creating new classes, reusing the properties of the existing ones.
- The mechanism of deriving a new class from an old one is called inheritance (or derivation).
- The old class referred to base class, parent class or super class and the new one is called the derived class, child class or sub class.
- The following table describes the visibility of inherited members:

Table 8.1 *Visibility of inherited members*

<i>Base class visibility</i>	<i>Derived class visibility</i>		
	<i>Public derivation</i>	<i>Private derivation</i>	<i>Protected derivation</i>
Private →	Not inherited	Not inherited	Not inherited
Protected →	Protected	Private	Protected
Public →	Public	Private	Protected

Derived class declaration

- A derived class can be defined by specifying its relationship with the base class in addition to its own details.
- The general form of defining a derived class is:

```

Class derived-class-name : visibility-mode base-class-name
{
    .....
    .....
}

```

};

- The colon indicates that the derived-class-name is derived from the base-class-name.
- The visibility-mode is optional and, if present, may be either private or public because private is default visibility mode.

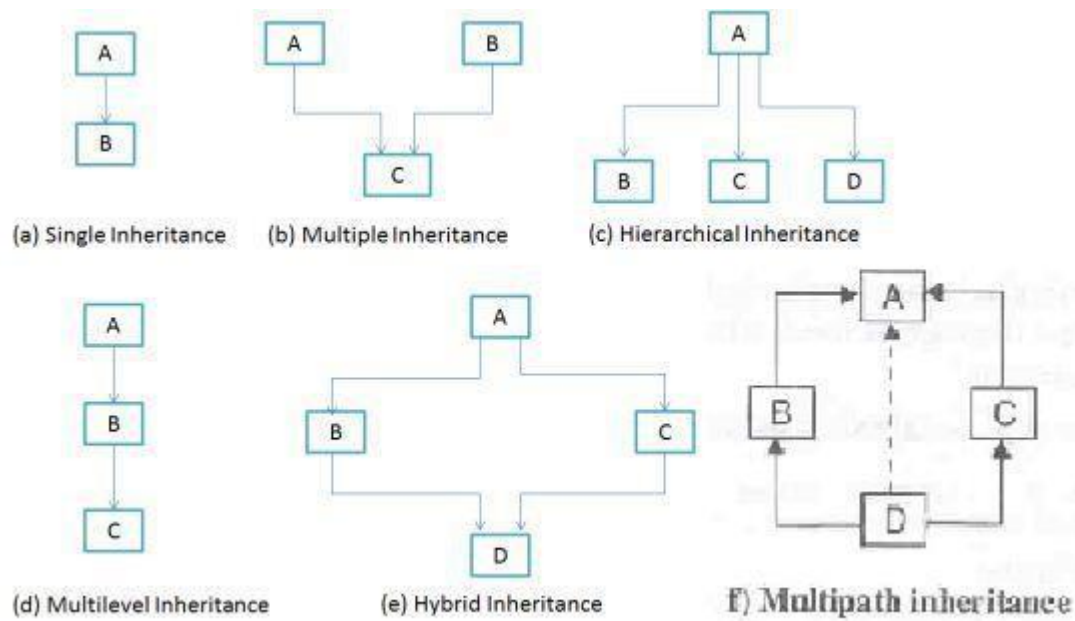
- Example

```
class ABC: private XYZ           //private derivation
{
    Member of ABC
};
class ABC: public XYZ           //public derivation
{
    Member of ABC
};
class ABC: XYZ                   //private derivation by default
{
    Member of ABC
};
```

- When a base class is privately inherited by a derived class, 'public member' of the base class become 'private member' of the derived class.
- Thus, the public members of the base class can only be accessed by the member functions of the derived class. They are inaccessible to the objects of the derived class.
- In inheritance, some of the base class elements and member functions are 'inherited' into the derived class.
- We can add our own data and functions and thus extend the functionality of the base class.
- Inheritance becomes a very powerful tool for incremental program development, when modify and extend the capabilities of the existing classes.

Forms of inheritance

- A derived class inherits some or all the features of the base class depending on the visibility mode and level of inheritance.
- Inheritance is classified into the following forms based on the levels of inheritance and interrelation among the classes involved in the inheritance process:



Single inheritance

- Derivation of a class from only one base class is called single inheritance.
- Example:

```
#include<iostream.h>
#include<conio.h>
Class a
{
    Int x,y;
    Public:
    Void getdata();
    Void putdata();
};
Void a::getdata();
{
    Cout<<"enter a X:";
    Cin>>x;
    Cout<<"enter a Y:";
    Cin>>y;
}
Void a::putdata()
{
    Cout<<"A number is:"<<x;
    Cout<<"B number is:"<<y;
}
Class b:public a
{
    Int c;
    Public:
    Void sum();
```

```

};

Void b::sum()
{
    C=a+b;
    Cout<<"sum is:"<<c;
}
Void main()
{
    Clrscr();
    B b1;
    B1.getdata();
    B1.putdata();
    B1.sum();
    Getch();
}

```

Multiple Inheritances

- Derivation of a class from several (two or more) base classes is called multiple inheritance.
- Example:

```

#include<iostream.h>
#include<conio.h>
Class n
{
    Int a;
    Void get_m(int x)
};
Void n::get_m(int x)
{
    M=x;
}
Class M
{
    Int b;
    Void get_n(int x);
};
Void M::gey_n(int x)
{
    N=x;
}
Class p:public n,public m
{
    Public:
    Void display();
};
Void p:display()
{

```

```

        Cout<<m;
        Cout<<n;
    }

    Void main()
    {
        P p1;
        P1.get_m(10);
        P1.get_n(10);
        P1.display();
        Getch();
    }

```

Hierarchical Inheritance

- Derivation of several class from a single base classes is called hierarchical inheritance.
- Example:

```

#include<iostream.h>
#include<conio.h>
Class cricketer
{
    Int id;
    Char name[15],team[15];
    Public:
        Void getcricket();
        Void putcricket();
};
Void crickerer::getcricket()
{
    Cin>>id;
    Cin>>name;
    Cin>>team;
}
Void cricketer::putcircket()
{
    Cout<<" cricketer id is:"<<id;
    Cout<<" cricketer name is:"<<name;
    Cout<<"cricketer team is:"<<team;
}
Class batsman:public cricketer
{
    Int match,runs,avg;
    Public:
        Void getbatsman();
        Void putbatsman();
        Void cal();
};
Void batsman::getbatsman()
{
    Cin>>match;
    Cin>>runs;
}

```



```

}
Void batsman::putbatsman()
{
    Cout<<"match is:"<<match;
    Cout<<"run is:"<<runs;
}
Void batsman::cal()
{
    Avg=runs/match;
}
Class bowler:public cricketer
{
    Int match,wicket,avg;
    Public:
        Void getbowler();
        Void putbowler();
        Void cal();
};
Void bowler::getbowler()
{
    Cin>match;
    Cin>>wicket;
}
Void bowler::putbowler()
{
    Cout<<"match is :"<<match;
    Cout<<"wicket is:"<<wicket;
    Cout<<"average is:"<<avg;
}
Void bowler::cal()
{
    Avg=wicket/match;
}
Void main()
{
    Clrscr();
    Batsman b;
    b.getcricketer();
    b.getbatsman();
    b.cal();
    b.putcricketer();
    b.putbatsman();

    bowler b1;
    b1.getcricketer();
    b1.getbowler();
    b1.cal();
    b1.putcricketer();
    b1.putbowler();
    getch();
}

```

Multilevel Inheritance

- Derivation of a class from another derived class is called multilevel inheritance.
- Example:

```
#include<iostream.h>
#include<conio.h>
Class student
{
    Int roll;
    Char name[15];
    Public:
        Void getdata();
        Void putdata();
};
Void student::getdata()
{
    Cin>>roll;
    Cin>>name;
}
Void student::putdata()
{
    Cout<<"roll no is:"<<roll;
    Cout<<"name is:"<<name;
}
Class test: public student
{
    Int m1,m2,m3;
    Public:
        Void get_makrs();
        Void put_marks();
};
Void test::get_marks()
{
    Cin>>m1;
    Cin>>m2;
    Cin>>m3;
}
Void test::put_marks()
{
    Cout<<"marks1 is:"<<m1;
    Cout<<"marks2 is:"<<m2;
    Cout<<"marks3 is:"<<m3;
}
Class result:public test
{
    Int avg,tot;
    Public:
        Void display();
};
```

Void result::display()

```

{
    Tot=m1+m2+m3;
    Cout<<"total is:"<<tot;
    Avg=tot/3;
    Cout<<"average is : "<<avg;
}
Void main()
{
    Clrscr();
    Result r;
    r.getdata();
    r.putdata();
    r.get_marks();
    r.put_marks();
    r.display();
    getch();
}

```

Inheritance and Member Accessibility

- Inheritance is the features that use the existing class without modifying them. The private member of a base class, which cannot be inherited, but if you want to inherit then it is declared as a protected.
- The following are rules for defining members as public, private or protected.
 - A private member is accessible only to members of the class in which the private member is declared. They cannot be inherited.
 - A private member of the base class can be accessed in the derived class through the member functions of the base class.
 - A protected member is accessible to members of its own class and to any of the members in a derived class.
 - If a class is expected to be used as a base class in future, then members which might be needed in the derived class should be declared protected rather than private.
 - A public member is accessible to members of its own class, members of derived class, and outside users of the class.
 - The private, protected and public sections may appear as many times as needed in a class and in any order.
 - The visibility mode in the derivation of a new class can be either private or public.
 - Constructors of the base class and derived class are automatically called when the object of a derived class is created. If a base class has parameterized constructor then it is called explicitly but default constructor are called explicitly. The constructor and destructor must

be defined in the public section of a class (base and derived) otherwise, the compiler generates the error.

Member functions accessibility

- The various categories of functions which have access to the private and protected members could be any of the following.
 - A member function of a class
 - A member function of a derived class
 - A friend function of a class
 - A member function of a friend class

Function Type	Access directly to		
	Private	Protected	Public
Class member	Yes	Yes	Yes
Derived class member	No	Yes	Yes
Friend	Yes	Yes	Yes
Friend class member	Yes	Yes	Yes

- The friend functions and member functions of a friend class have direct access to both the private and protected members of a class.

Function Overriding

As we know, inheritance is a feature of OOP that allows us to create derived classes from a base class. The derived classes inherit features of the base class. Suppose, the same function is defined in both the derived class and the based class. Now if we call this function using the object of the derived class, the function of the derived class is executed. This is known as function overriding in C++. The function in derived class overrides the function in base class.

```
#include <iostream>
using namespace std;
```

```
class Base
{
    public:
```

```

void print() {
    cout << "Base Function" << endl;
}

};

class Derived : public Base
{
public:
    void print() {
        cout << "Derived Function" << endl;
    }
};

int main()
{
    Derived derived1;
    derived1.print();
    return 0;
}

```

Output

Derived Function

Here, the same function `print()` is defined in both Base and Derived classes. So, when we call `print()` from the Derived object `derived1`, the `print()` from Derived is executed by overriding the function in Base.

```

class Base {
public:
    void print() {
        // code
    }
};

class Derived : public Base {
public:
    void print() { ←
        // code
    }
};

int main() {
    Derived derived1;
    derived1.print(); ←
    return 0;
}

```



Working of function overriding in C++

As we can see, the function was overridden because we called the function from an object of the Derived class. Had we called the print() function from an object of the Base class, the function would not have been overridden.

```
// Call function of Base class
Base base1;
base1.print(); // Output: Base Function
```

Access Overridden Function in C++

To access the overridden function of the base class, we use the scope resolution operator ::

We can also access the overridden function by using a pointer of the base class to point to an object of the derived class and then calling the function from that pointer.

```
#include <iostream>
using namespace std;

class Base
{
public:
    void print() {
        cout << "Base Function" << endl;
    }
};

class Derived : public Base
{
public:
    void print() {
        cout << "Derived Function" << endl;
    }
};

int main()
{
    Derived derived1, derived2;
    derived1.print();

    // access print() function of the Base class
    derived2.Base::print();

    return 0;
}
```

Output

Derived Function

Base Function

Here, this statement

`derived2.Base::print();`
accesses the `print()` function of the Base class.

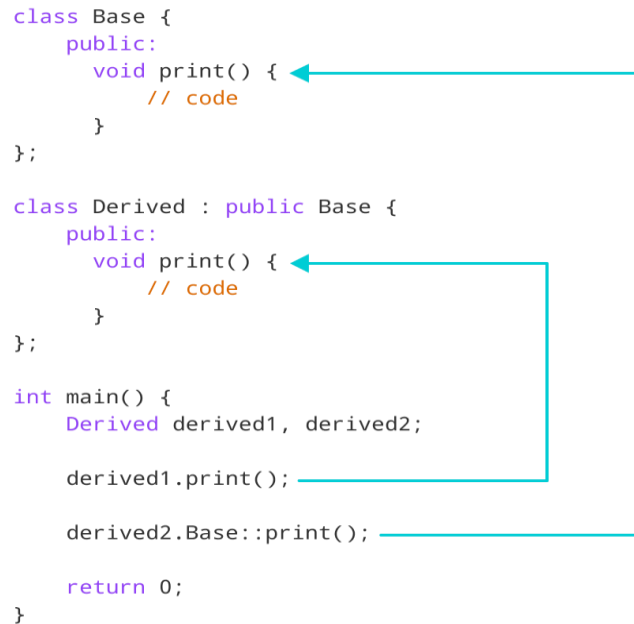
```
class Base {
public:
    void print() { // code
    }
};

class Derived : public Base {
public:
    void print() { // code
    }
};

int main() {
    Derived derived1, derived2;

    derived1.print();
    derived2.Base::print();

    return 0;
}
```



Access overridden function using object of derived class in C++ **Call Overridden Function from Derived Class**

// C++ program to call the overridden function
// from a member function of the derived class

```
#include <iostream>
using namespace std;
```

```
class Base
{
public:
    void print()
    {
        cout << "Base Function" << endl;
    }
};

class Derived : public Base
{
public:
    void print()
    {
        cout << "Derived Function" << endl;
    }
}
```

```

        // call overridden function
        Base::print();
    }
};

int main()
{
    Derived derived1;
    derived1.print();
    return 0;
}

```

Output

Derived Function
Base Function

In this program, we have called the overridden function inside the Derived class itself.

```

class Derived : public Base
{
    public:
    void print()
    {   cout << "Derived Function" << endl;
        Base::print();
    }
};

```

Notice the code `Base::print();`, which calls the overridden function inside the Derived class.

```

class Base {
public:
    void print() { ←
        // code
    }
};

class Derived : public Base {
public:
    void print() {
        // code
        Base::print(); →
    }
};

int main() {
    Derived derived1;
    derived1.print();
    return 0;
}

```

The diagram illustrates the execution flow. A red arrow points from the `derived1.print();` line in the `main()` function to the `void print()` line in the `Derived` class. Another red arrow points from the `Base::print();` line inside the `Derived::print()` function to the `void print()` line in the `Base` class, indicating that the `Derived` class calls the `print` function of its base class.

Access overridden function inside derived class in C++

Virtual Functions

- Polymorphism in biology means the ability of an organism to assume a variety of forms. In c++, it indicates the form of a member function that can be changed at runtime. Such member functions are called virtual functions.
- When we use the same function name in both the base and derived classes, the function in base class is declared as virtual using the keyword **virtual** preceding its normal declaration.
- When a function is made virtual, c++ determines which function to use at run time based in the type of object pointed to by the base pointer, rather than the type of the pointer. Thus, by making the base pointer to point to different objects. We can execute different versions of the virtual function.
- Example:

```
#include<iostream.h>
class base
{
    Public:
        Void display()
        {
            Cout<<"\n display base";
        }
        Virtual void show()
        {
            Cout<<"\n show base";
        }
};
Class derived : public base
{
    Public:
        Void display()
        {
            Cout<<"\n display derived";
        }
        void show()
        {
            Cout<<"\n show derived";
        }
};
Int main()
{
    Base b;
    Derived d;
    Base *bptr;

    Cout<<"\n bptr points to base";
    Bptr=&b;
    Bptr -> display();           //calls base version
    Bptr -> show();             //calls base version
}
```

```

Cout<<"\n bptr points to derived";
Bptr=&d;
Bptr -> display();           //calls base version
Bptr -> show();              //calls derived version

```

```

Return 0;

```

```

}

```

- **Note :** when bptr is made to point to the object d, the statement

```

Bptr -> display();

```

Calls only the function associated with the base (i.e. base:: display()), whereas the statement

```

Bptr -> show();

```

Calls the derived version of show().

This is because the function display() has not been made virtual in the base class

- It means, if you want to access function with object of derived class which is same function name as base class, you have to declare base class function as virtual otherwise, it will call base class function first.

Pure Virtual Functions

Pure virtual functions are used. if a function doesn't have any use in the base class but the function must be implemented by all its derived classes

Let's take an example, suppose, we have derived Triangle, Square and Circle classes from the Shape class, and we want to calculate the area of all these shapes. In this case, we can create a pure virtual function named calculateArea() in the Shape. Since it's a pure virtual function, all derived classes Triangle, Square and Circle must include the calculateArea() function with implementation. A pure virtual function doesn't have the function body and it must end with = 0. For example,

```

class Shape
{
public:
    // creating a pure virtual function
    virtual void calculateArea() = 0;
};

```

Note: The = 0 syntax doesn't mean we are assigning 0 to the function. It's just the way we define pure virtual functions.

Abstract Class

A class that contains a pure virtual function is known as an abstract class. In the above example, the class Shape is an abstract class. We cannot create objects of an abstract class. However, we can derive classes from them, and use their data members and member functions (except pure virtual functions).

Abstract Class and Pure Virtual Function

```
// C++ program to calculate the area of a square and a circle
```

```
#include <iostream>
using namespace std;
```

```
// Abstract class
class Shape
{
protected:
    float dimension;

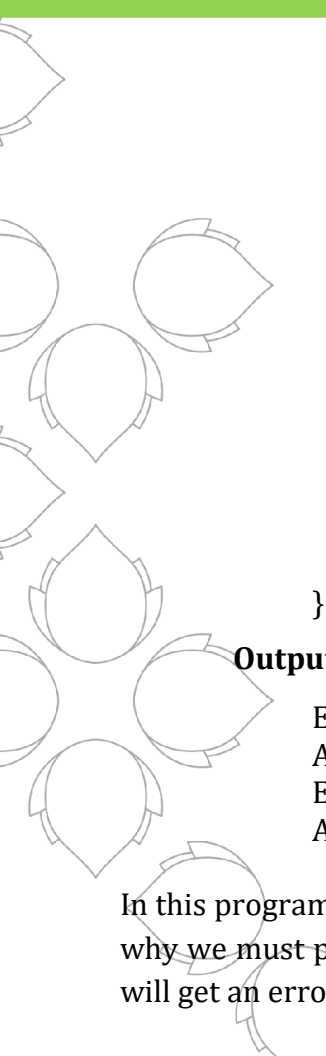
public:
    void getDimension()
    {
        cin >> dimension;
    }
}
```

```
// pure virtual Function
virtual float calculateArea() = 0;
};
```

```
// Derived class
class Square : public Shape
{
public:
    float calculateArea() {
        return dimension * dimension;
    }
};
```

```
// Derived class
class Circle : public Shape
{
public:
    float calculateArea() {
        return 3.14 * dimension * dimension;
    }
};
```

```
int main()
{
```



```
Square square;  
Circle circle;
```

```
cout << "Enter the length of the square: ";  
square.getDimension();  
cout << "Area of square: " << square.calculateArea() << endl;
```

```
cout << "\nEnter radius of the circle: ";  
circle.getDimension();  
cout << "Area of circle: " << circle.calculateArea() << endl;
```

```
return 0;  
}
```

Output

```
Enter the length of the square: 4  
Area of square: 16  
Enter radius of the circle: 5  
Area of circle: 78.5
```

In this program, virtual float calculateArea() = 0; inside the Shape class is a pure virtual function. That's why we must provide the implementation of calculateArea() in both of our derived classes, or else we will get an error.